



การจัดการ Image

ว30265 การประเมินผลภาพ

Image Resizing

การปรับขนาดรูป ด้วย opencv จะต้องใช้คำสั่ง `cv.resize()` ในการปรับขนาดรูปตามความกว้างความสูงที่กำหนด



```
1 cv.resize(src, dsize[, dst[, fx[, fy[, interpolation]]])
```

Image Resizing

src: เป็นรูปภาพอินพุตที่ต้องการ ซึ่งอาจเป็นสตริงที่มีเส้นทางของรูปภาพอินพุต (เช่น: 'test_image.png')

dsize: เป็นขนาดที่ต้องการของภาพที่ส่งออก โดยสามารถกำหนดความสูงและความกว้างใหม่ได้

fx: ตัวประกอบสเกลตามแกนแนวนอน

fy: ตัวประกอบสเกลตามแกนตั้ง

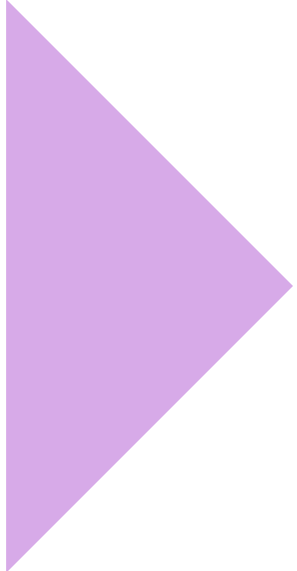
interpolation: ทางเลือกในการปรับขนาดรูปภาพด้วยวิธีต่างๆ



```
1 cv.resize(src, dsize[, dst[, fx[, fy[, interpolation]]])
```

Image Resizing

```
1  import cv2
2  import numpy as np
3
4  image = cv2.imread('image.jpg')
5  cv2.imshow('Original Image', image)
6
7  down_width = 300
8  down_height = 200
9  down_points = (down_width, down_height)
10 resized_down = cv2.resize(image, down_points, interpolation= cv2.INTER_LINEAR)
11
12 up_width = 600
13 up_height = 400
14 up_points = (up_width, up_height)
15 resized_up = cv2.resize(image, up_points, interpolation= cv2.INTER_LINEAR)
16
17 cv2.imshow('Resized Down by defining height and width', resized_down)
18 cv2.waitKey()
19 cv2.imshow('Resized Up image by defining height and width', resized_up)
20 cv2.waitKey()
21
22 cv2.destroyAllWindows()
```



สามารถ Resizing และยังสามารถ
รักษาอัตราส่วนของภาพได้
อย่างไร

Image Resizing

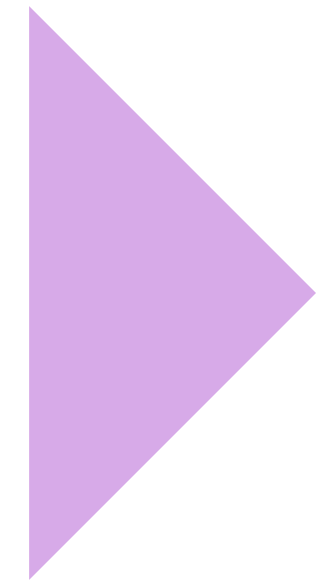


```
1 h,w,c = image.shape
2 print("Original Height and Width:", h,"x", w)
```

Image Resizing



```
1 # Scaling Up the image 1.2 times by specifying both scaling factors
2 scale_up_x = 1.2
3 scale_up_y = 1.2
4 # Scaling Down the image 0.6 times specifying a single scale factor.
5 scale_down = 0.6
6
7 scaled_f_down = cv2.resize(image, None, fx= scale_down, fy= scale_down, interpolation= cv2.INTER_LINEAR)
8 scaled_f_up = cv2.resize(image, None, fx= scale_up_x, fy= scale_up_y, interpolation= cv2.INTER_LINEAR)
9
```



Cropping an Image

Cropping an Image

ในการครอบหรือตัดรูปใน opencv จะใช้การเลือกข้อมูลมาใช่ว่าจะใช้ส่วนไหนบ้าง



```
1  cropped = img[start_row:end_row, start_col:end_col]
```



```
1  import cv2
2  import numpy as np
3
4  img = cv2.imread('test.jpg')
5  print(img.shape)
6  cv2.imshow("original", img)
7
8  cropped_image = img[80:280, 150:330]
9
10 cv2.imshow("cropped", cropped_image)
11
12 cv2.imwrite("Cropped Image.jpg", cropped_image)
13
14 cv2.waitKey(0)
15 cv2.destroyAllWindows()
```

Cropping an Image

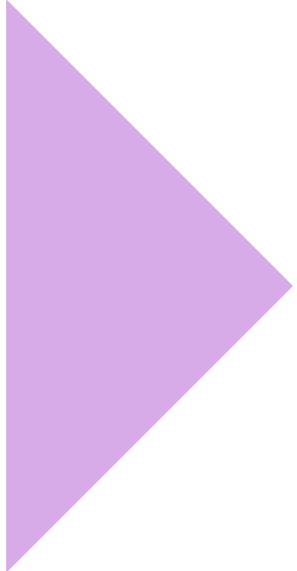


Image Rotation and Translation



```
1 cv.rotate('imagenam', rotateCode)
2
3 # rotateCode
4 cv.ROTATE_90_CLOCKWISE
5 cv.ROTATE_180
6 cv.ROTATE_90_COUNTERCLOCKWISE
```

Image Rotation

Image Rotation

ในการหมุนมุมอื่นๆ จะต้องใช้ การคูณเมทริกซ์ โดยถ้าให้รูปหนึ่งอยู่ที่พิกัด $x\ y$ จะเขียนเมทริกซ์ได้ว่า

$$\begin{pmatrix} x_1 \\ y_1 \\ 1 \end{pmatrix}$$

$$M = \begin{pmatrix} a_x & b_x & c_x \\ a_y & b_y & c_y \end{pmatrix}$$

โดยค่า 1 คือค่า z เพราะรูปอยู่ในรูปแบบ 2 มิติ

ถ้ามีจำนวน n ตัวจะเขียนได้ในรูป

Image Rotation

เมทริกซ์ที่ใช้บิดแปลงภาพด้วย `cv2.warpAffine()` คือเมทริกซ์ขนาด 2×3

$$x = \begin{pmatrix} x_1 & x_2 & \dots & x_n \\ y_1 & y_2 & \dots & y_n \\ 1 & 1 & \dots & 1 \end{pmatrix}$$

Image Rotation

เมทริกซ์ของพิกัดที่จะได้หลังแปลงนั้นจะได้มาจากการเอาเมทริกซ์การแปลงมาคูณเข้ากับเมทริกซ์พิกัดเดิม

$$\begin{aligned}\vec{X}_* &= \vec{M} \times \vec{X} \\ &= \begin{bmatrix} a_x & b_x & c_x \\ a_y & b_y & c_y \end{bmatrix} \times \begin{bmatrix} x_1 & x_2 & \cdots & x_n \\ y_1 & y_2 & \cdots & y_n \\ 1 & 1 & \cdots & 1 \end{bmatrix} \\ &= \begin{bmatrix} a_x x_1 + b_x y_1 + c_x & a_x x_2 + b_x y_2 + c_x & \cdots & a_x x_n + b_x y_n + c_x \\ a_y x_1 + b_y y_1 + c_y & a_y x_2 + b_y y_2 + c_y & \cdots & a_y x_n + b_y y_n + c_y \\ 1 & 1 & \cdots & 1 \end{bmatrix}\end{aligned}$$

Image Rotation

ก็จะได้เมทริกซ์ของพิกัดใหม่

$$\vec{X}_* = \begin{bmatrix} x_{1*} & x_{2*} & \cdots & x_{n*} \\ y_{1*} & y_{2*} & \cdots & y_{n*} \\ 1 & 1 & \cdots & 1 \end{bmatrix}$$

Image Rotation

ค่าพิกัดใหม่ในแต่ละจุดเป็น

$$x_{i*} = a_x x_i + b_x y_i + c_x$$

$$y_{i*} = a_y x_i + b_y y_i + c_y$$

การใช้ `cv2.warpAffine()` จะเป็นการ
ย้ายจุดทุกจุดพิกเซลบนภาพจากพิกัด
เก่าไปพิกัดใหม่

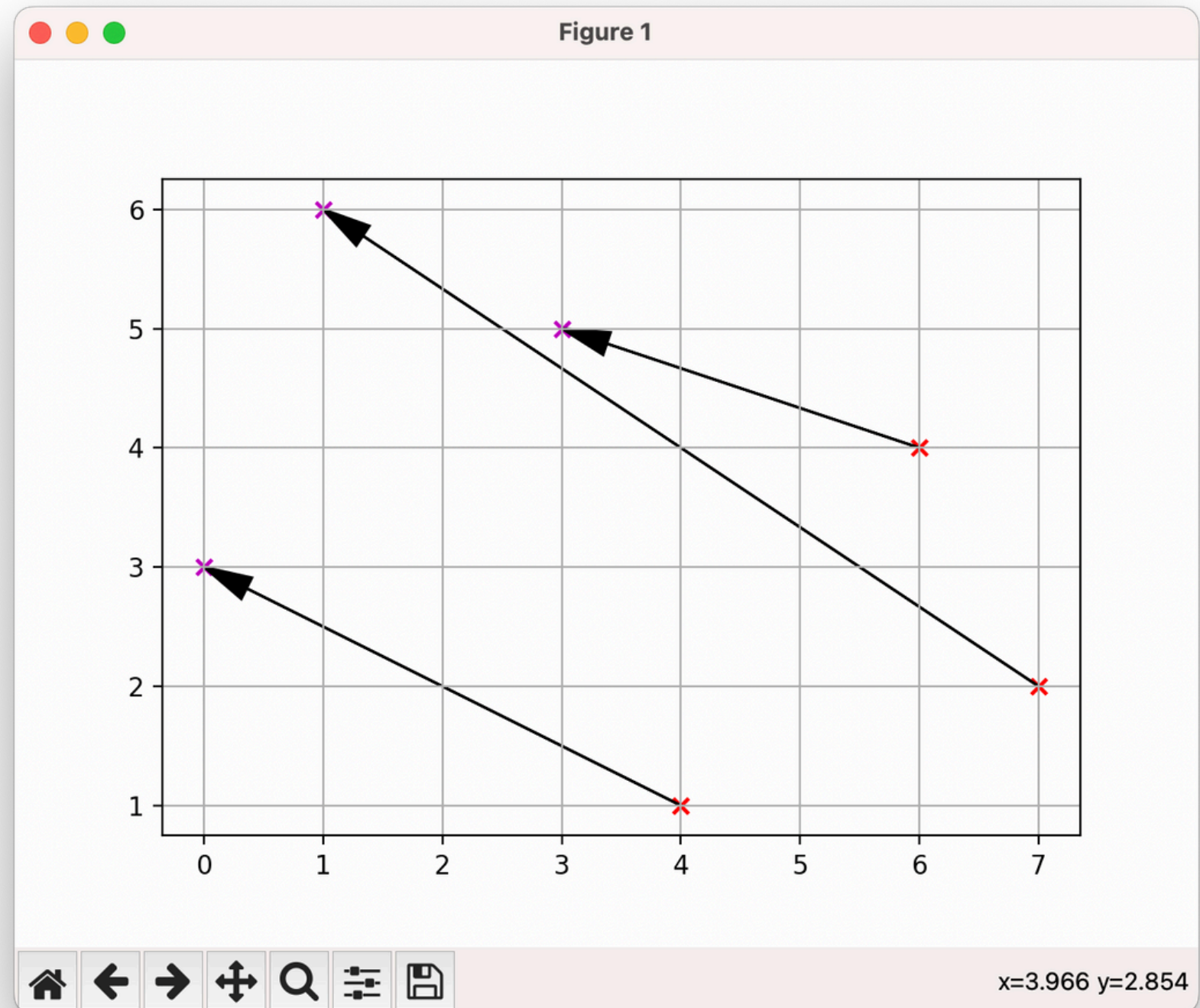


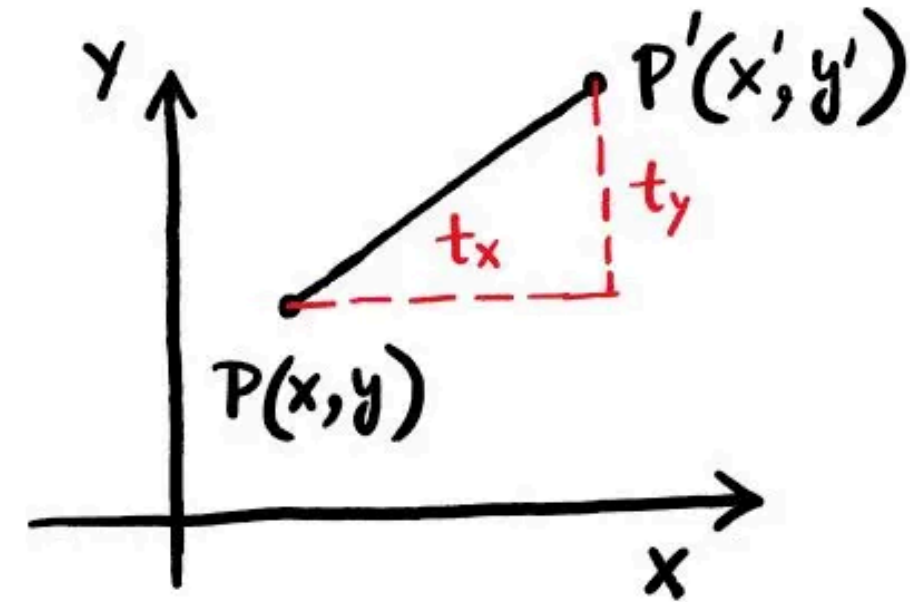
Image Rotation



```
1 cv2.warpAffine(src, M, dsize[, dst[, flags[, borderMode[, borderValue]]]])
2
3 src: ภาพต้นฉบับ
4 M: เมทริกซ์การแปลง
5 dsize: ขนาดของภาพผลลัพธ์
6 dst: ภาพผลลัพธ์
7 flags: การรวมกันของวิธีการแทรกค่า เช่น INTER_LINEAR หรือ INTER_NEAREST
8 borderMode: วิธีการประมาณค่าพิกเซลนอกขอบภาพ
9 borderValue: ค่าที่จะใช้ในกรณีที่ใช้ border แบบคงที่ มีค่าเริ่มต้นเป็น 0
10
```

```
1 mat = np.float32([[0.6,0,0],
2                   [0,0.4,0]])
3 plt.imshow(cv.warpAffine(img,mat,(1280,797))[:,::-1])
4 plt.tight_layout()
5 plt.show()
```

$$\vec{M} = \begin{bmatrix} \text{จำนวนเท่าแนวนอน} & 0 & 0 \\ 0 & \text{จำนวนเท่าแนวตั้ง} & 0 \end{bmatrix}$$



we can write

$$x' = x + t_x$$

$$y' = y + t_y$$

In matrix form

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

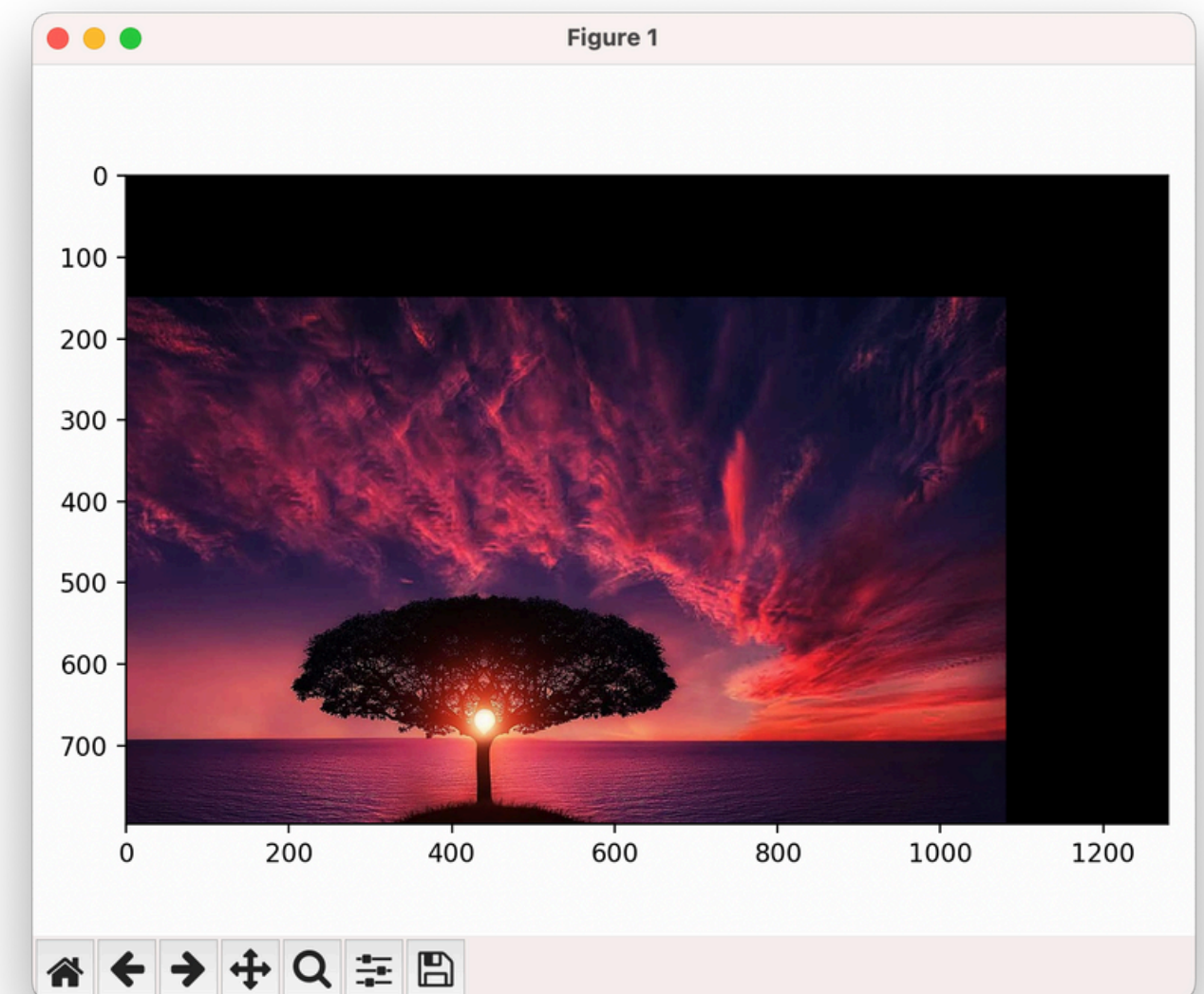
Transformation
Matrix (M)



```
1 mat = np.float32([[0.6,0,0],  
2                  [0,0.4,0]])  
3 plt.imshow(cv.warpAffine(img,mat,(1280,797))[:,::-1])  
4 plt.tight_layout()  
5 plt.show()
```

$$\vec{M} = \begin{bmatrix} 1 & 0 & \text{ระยะเลื่อนแนวนอน} \\ 0 & 1 & \text{ระยะเลื่อนแนวตั้ง} \end{bmatrix}$$

การเลื่อนตำแหน่ง

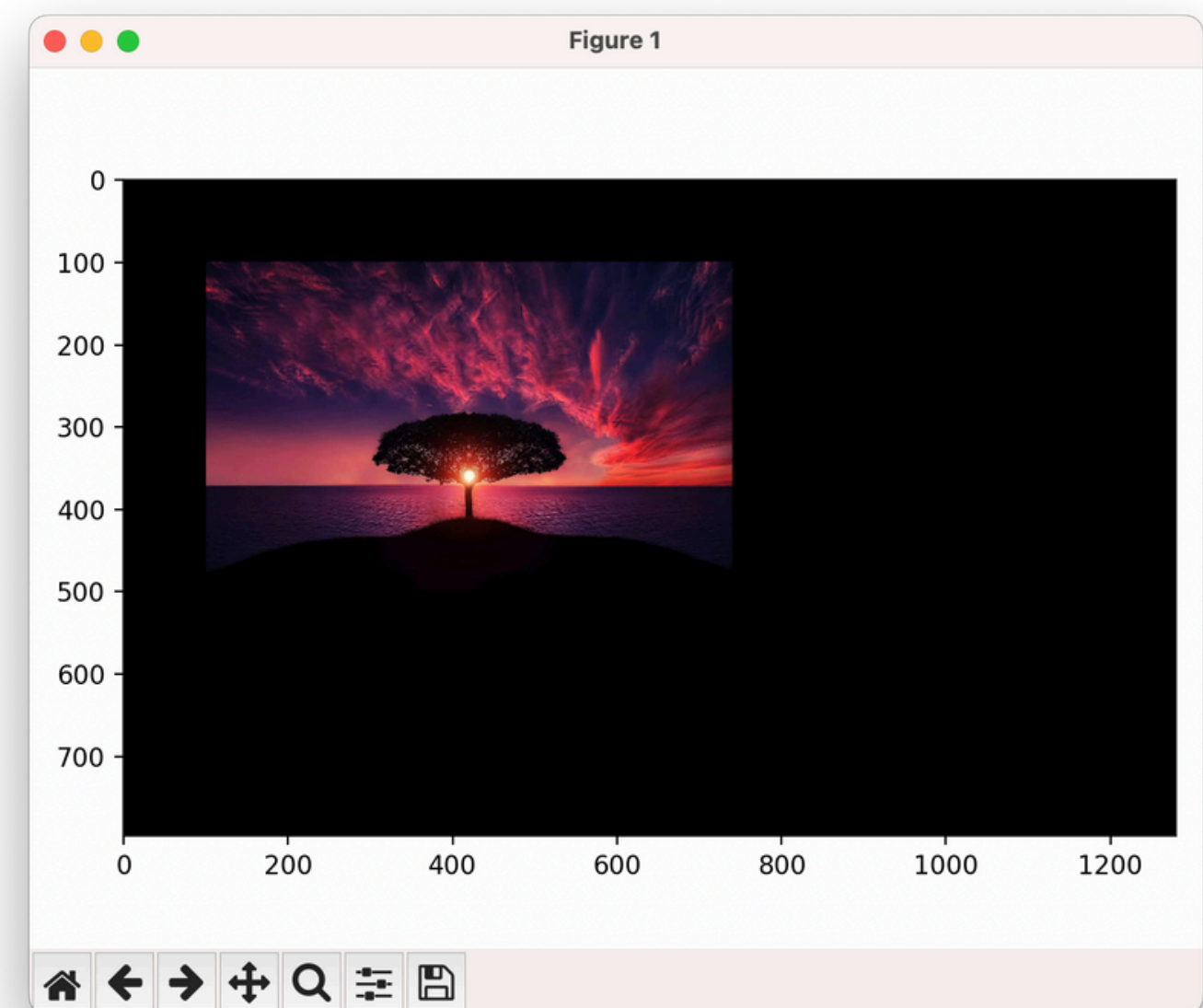




```
1 mat = np.float32([[0.5,0,100],  
2                  [0,0.5,100]])  
3 plt.imshow(cv.warpAffine(img,mat,(1280,797))[:, :, ::-1])  
4 plt.tight_layout()  
5 plt.show()
```

$$\vec{M} = \begin{bmatrix} \text{จำนวนเท่าแนวนอน} & 0 & \text{ระยะเลื่อนแนวนอน} \\ 0 & \text{จำนวนเท่าแนวตั้ง} & \text{ระยะเลื่อนแนวตั้ง} \end{bmatrix}$$

ทั้งเลื่อนตำแหน่งและย่อขยาย



การหมุนภาพ

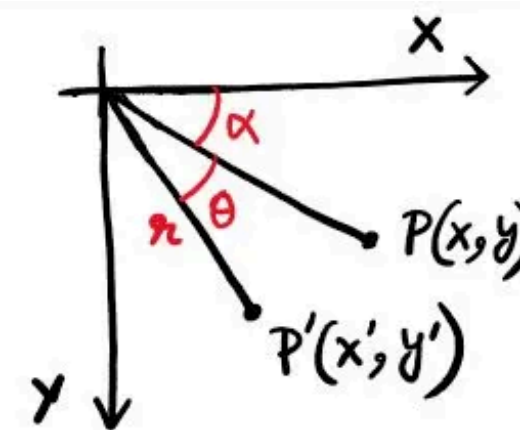
จะต้องใช้ เมทริกซ์

$$\vec{M} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \end{bmatrix}$$

โดย θ คือขนาดของมุมที่ต้องการ
หมุนภาพ ตามเข็มนาฬิกา

we can write

$$\begin{aligned} x &= r \cos\alpha \\ y &= r \sin\alpha \end{aligned}$$



Similarly we can write

$$\begin{aligned} x' &= r \cos(\alpha + \theta) \\ x' &= r \cos\alpha \cos\theta - r \sin\alpha \sin\theta \\ x' &= x \cos\theta - y \sin\theta \end{aligned}$$

$$\begin{aligned} y' &= r \sin(\alpha + \theta) \\ y' &= r \sin\alpha \cos\theta + r \cos\alpha \sin\theta \\ y' &= y \cos\theta + x \sin\theta \end{aligned}$$

So, in the matrix form we can write

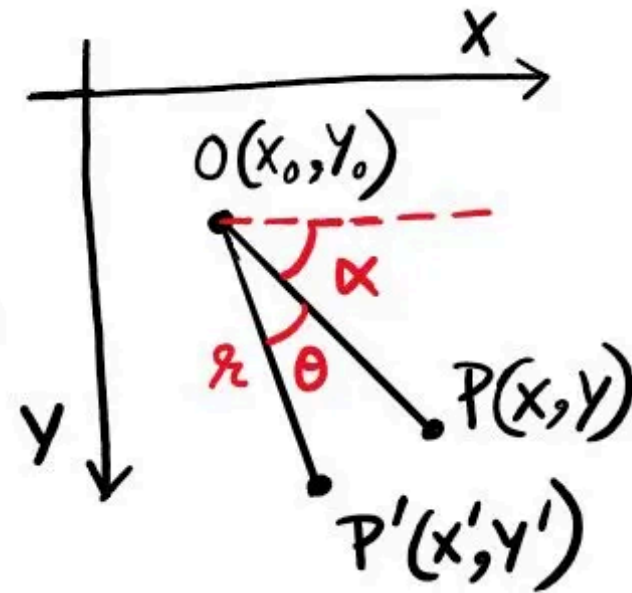
$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Transformation
Matrix M

we can write

$$x = x_0 + r \cos \alpha \rightarrow \textcircled{1}$$

$$y = y_0 + r \sin \alpha \rightarrow \textcircled{2}$$



Similarly, we can write

$$\begin{aligned} x' &= x_0 + r \cos(\alpha + \theta) \\ &= x_0 + r \cos \alpha \cos \theta - r \sin \alpha \sin \theta \end{aligned}$$

Using 1 and 2 from above

$$= x_0 + (x - x_0) \cos \theta - (y - y_0) \sin \theta$$

$$x' = x \cos \theta - y \sin \theta + x_0(1 - \cos \theta) + y_0 \sin \theta$$

Similarly

$$y' = y_0 + r \sin(\alpha + \theta)$$

$$y' = x \sin \theta + y \cos \theta + y_0(1 - \cos \theta) - x_0 \sin \theta$$

In matrix form we can write

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & x_0(1 - \cos \theta) + y_0 \sin \theta \\ \sin \theta & \cos \theta & y_0(1 - \cos \theta) - x_0 \sin \theta \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

การหมุนภาพ

เมื่อไม่ได้หมุนที่จุด Center

$$\vec{M} = \begin{bmatrix} \cos \theta & -\sin \theta & c_x(1 - \cos \theta) - c_y \sin \theta \\ \sin \theta & \cos \theta & c_x \sin \theta + c_y(1 - \cos \theta) \end{bmatrix}$$

โดย θ คือขนาดของมุมที่ต้องการ
หมุนภาพ ตามเข็มนาฬิกา

การหมุนภาพ

แต่ว่าใน `opencv` ได้เตรียมฟังก์ชันสำหรับหาเมทริกซ์แบบนั้นให้โดยอัตโนมัติโดยไม่จำเป็นต้องเขียนเอง นั่นคือ `cv2.getRotationMatrix2D()`



```
1 cv2.getRotationMatrix2D(center, angle, scale)
```



```
1 rotate_matrix = cv.getRotationMatrix2D(center=center, angle=45, scale=1)
2 rotated_image = cv.warpAffine(src=img, M=rotate_matrix, dsize=(width, height))
3
```

การแสดงผล

